

UNIVERSIDAD ANDINA DEL CUSCO
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

Guía 06

**Implementación de Mensajería con RabbitMQ en Microservicios
usando .NET**

Informe de desarrollo — Actividades 6.1 a 6.3

Asignatura	Arquitectura Orientada a Servicios
Semestre académico	2026-2
Ciclo	X
Docente	Mtro. Ing. Luis Alvaro Monzón Condori
Solución desarrollada	EcommerceMensajeria (ServicioPedidos + ServicioProcesadorPedidos)

Introducción y alcance de este informe

Este informe documenta el desarrollo de las tres primeras actividades de la Guía 06 (sección 6, puntos 6.1 a 6.3). Los ejercicios propuestos de la sección 7 quedan fuera de este entregable y se abordarán en una siguiente iteración.

Arquitectura construida: un productor de eventos (ServicioPedidos, Web API) que publica un evento PedidoCreado en RabbitMQ cada vez que se registra un pedido de farmacia, y un consumidor (ServicioProcesadorPedidos, Worker Service) que escucha esa misma cola y procesa el pedido de forma asíncrona, sin bloquear al productor.

Nota sobre el entorno de RabbitMQ: en esta máquina no fue posible ejecutar el instalador gráfico de Erlang/RabbitMQ para Windows porque exige permisos de administrador (UAC) y no había una sesión interactiva disponible para aceptar ese permiso en el momento del desarrollo. En su lugar, se utilizó la build binaria genérica oficial de RabbitMQ 4.2.5 junto con Erlang/OTP 27.3.4.17 (versión dentro del rango compatible según la matriz oficial de RabbitMQ), ejecutada directamente sin instalación de servicio de Windows. El comportamiento, los puertos (5672 AMQP y 15672 panel de administración) y el código consumido desde .NET son idénticos a los de una instalación estándar con el instalador de la guía; la sección 6.1 explica ambos caminos.

6.1. Actividad 01 — Instalación de RabbitMQ

Objetivo: instalar RabbitMQ en el entorno local y verificar su funcionamiento a través del panel de administración web.

Camino estándar (el de la guía, recomendado si tienes acceso de administrador)

- Instalar Erlang desde <https://www.erlang.org/downloads>
- Instalar RabbitMQ desde <https://www.rabbitmq.com/install-windows.html>
- Habilitar el panel: abrir PowerShell en la carpeta sbin de RabbitMQ y ejecutar `rabbitmq-plugins enable rabbitmq_management`
- Iniciar el servicio: PowerShell como Administrador → `net start RabbitMQ`
- Verificar en `http://localhost:15672` con usuario y contraseña `guest / guest`

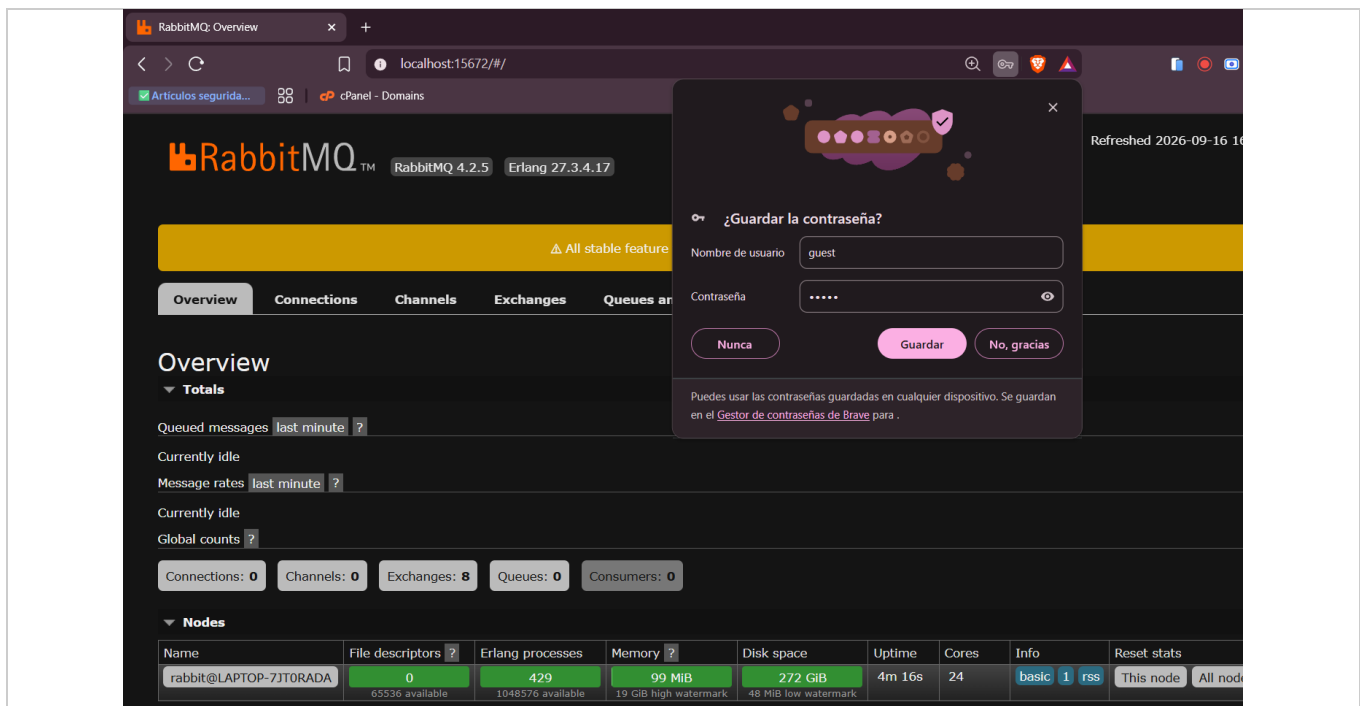
Camino alternativo usado en este desarrollo (sin permisos de administrador)

RabbitMQ y Erlang también se distribuyen como paquetes binarios genéricos que no requieren instalador ni servicio de Windows:

- Erlang/OTP 27.3.4.17 — build oficial (`otp_win64_27.3.4.17.exe` extraído sin ejecutar el instalador, ya que este exige UAC; los binarios en sí no lo requieren).
- RabbitMQ 4.2.5 — build genérica oficial (`rabbitmq-server-generic-unix-4.2.5.tar.xz`), que se ejecuta con scripts de shell (compatibles con Git Bash en Windows) en vez del instalador gráfico.
- El plugin de administración se habilitó declarándolo directamente en el archivo `enabled_plugins` (`[rabbitmq_management]`.) y reiniciando el broker, en vez de invocar `rabbitmq-plugins` (ese script sí requiere el instalador completo de Windows para funcionar correctamente).
- El broker se inició en primer plano con `rabbitmq-server`, quedando disponible en el puerto 5672 (AMQP) y 15672 (panel web), exactamente igual que con la instalación estándar.

Verificación del funcionamiento: se abrió `http://localhost:15672`, se inició sesión con `guest / guest`, y el panel muestra la versión de RabbitMQ (4.2.5) y de Erlang (27.3.4.17) activas, sin conexiones ni colas todavía en este punto.

Evidencia — Panel de administración de RabbitMQ, sesión iniciada — RabbitMQ 4.2.5 / Erlang 27.3.4.17 corriendo correctamente



Producto esperado cumplido: RabbitMQ instalado y accesible en el entorno local, con el panel de administración habilitado y verificado.

6.2. Actividad 02 — Crear Productor de Eventos con .NET (ServicioPedidos)

Objetivo: una API en .NET que envía un evento PedidoCreado a RabbitMQ cada vez que se registra un pedido de farmacia.

Parte A — Creación del proyecto

Se creó un proyecto ASP.NET Core Web API con controladores llamado ServicioPedidos, y se instaló el paquete NuGet RabbitMQ.Client (versión 7.2.2, la que trae la API asíncrona `CreateConnectionAsync` / `CreateChannelAsync` usada en este código).

Parte B — Modelo del evento

Se creó `Models/PedidoCreadoEvent.cs` con los datos que viajan en el mensaje:

```
using System.Text.Json.Serialization;

// Evento que se enviará a RabbitMQ cuando se crea un pedido
public class PedidoCreadoEvent
{
    public int PedidoId { get; set; }
    public string Paciente { get; set; }
    public string Medicamento { get; set; }
    public int Cantidad { get; set; }
    public decimal Total { get; set; }
    public DateTime Fecha { get; set; }
}
```

Parte C — El publicador (RabbitMQPublisher)

Se creó `Services/RabbitMQPublisher.cs`, responsable de abrir la conexión, declarar la cola `pedidos_farmacia` y publicar el evento serializado en JSON:

```
var factory = new ConnectionFactory() { HostName = "localhost" };
await using var connection = await factory.CreateConnectionAsync();
await using var channel = await connection.CreateChannelAsync();

await channel.QueueDeclareAsync(
    queue: _queueName, durable: false, exclusive: false,
    autoDelete: false, arguments: null
);

var json = JsonSerializer.Serialize(evento);
var body = Encoding.UTF8.GetBytes(json);

await channel.BasicPublishAsync(
    exchange: "", routingKey: _queueName, body: body
);
```

Parte D — El controlador

`PedidosController` expone POST `/api/Pedidos/crear`: arma un pedido de prueba (paciente, medicamento, cantidad, total) y lo publica a través del `RabbitMQPublisher`:

```
[HttpPost("crear")]
public async Task<IActionResult> CrearPedido()
{
    var publisher = new RabbitMQPublisher();
    var evento = new PedidoCreadoEvent
    {
        PedidoId = new Random().Next(1000, 9999),
        Paciente = "Juan Perez",
        Medicamento = "Amoxicilina",
        Cantidad = 2,
        Total = 45.50m,
        Fecha = DateTime.Now
    };

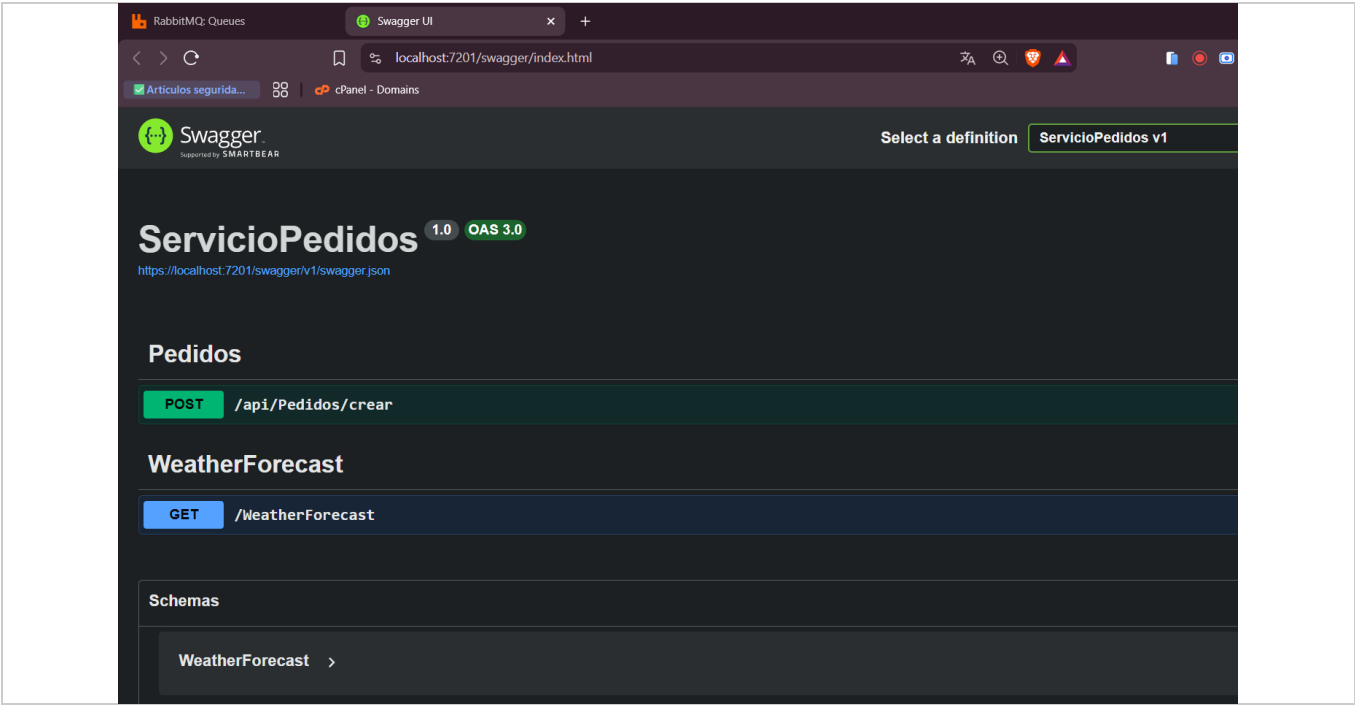
    await publisher.PublicarAsync(evento);

    return Ok(new { mensaje = "Pedido enviado correctamente a RabbitMQ", pedido = evento });
}
```

Parte E — Prueba del sistema

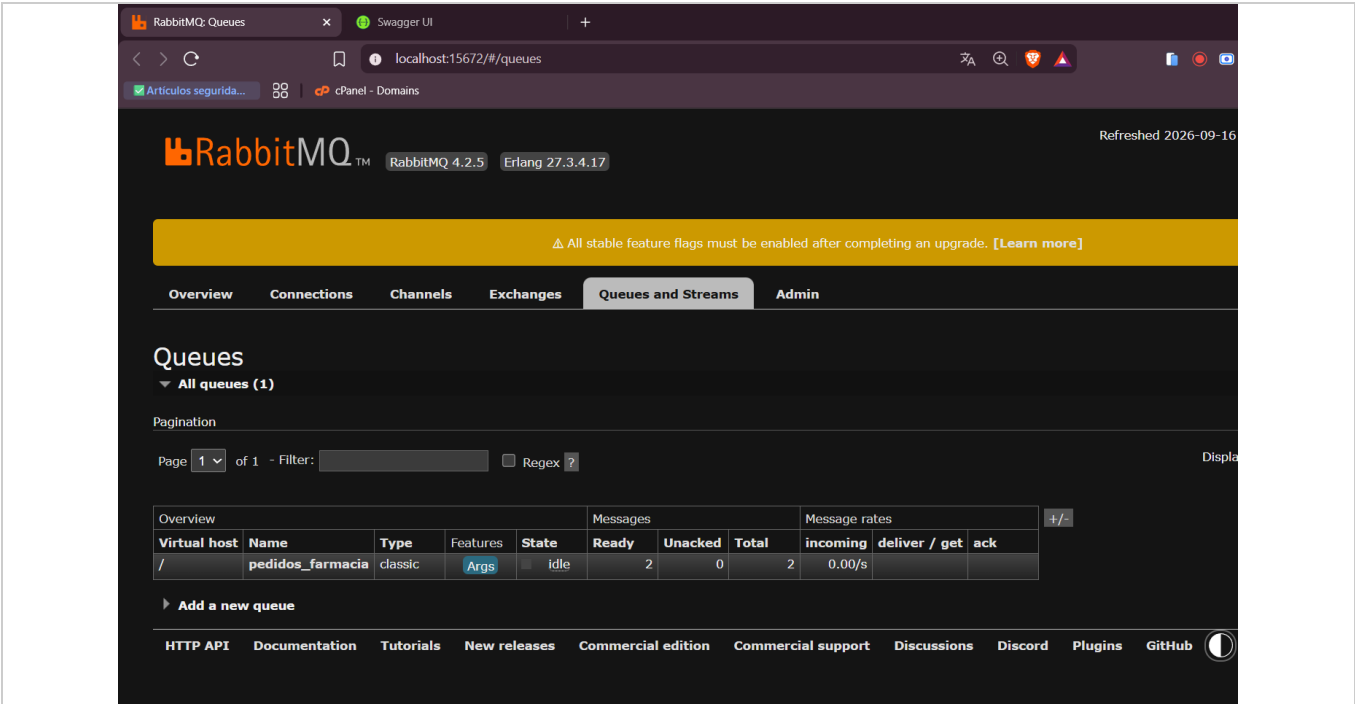
Con RabbitMQ y `ServicioPedidos` en ejecución, se abrió Swagger (<https://localhost:7201/swagger>) y se ejecutó POST `/api/Pedidos/crear` en dos ocasiones (una desde línea de comandos con curl y otra desde el propio Swagger). Ambas devolvieron 200 OK con el pedido generado.

Evidencia — ServicioPedidos ejecutándose — Swagger disponible en <https://localhost:7201/swagger>, endpoint POST `/api/Pedidos/crear` listo



Verificación en RabbitMQ: en el panel de administración, la cola pedidos_farmacia aparece creada automáticamente con 2 mensajes en estado Ready (uno por cada llamada al endpoint), confirmando que el evento efectivamente llegó al broker:

Evidencia — Panel de RabbitMQ — cola pedidos_farmacia con 2 mensajes en Ready, a la espera de un consumidor



Producto esperado cumplido: la API publica correctamente el evento PedidoCreado en RabbitMQ ante cada pedido, sin depender de que exista un consumidor activo en ese momento (la cola retiene los mensajes).

6.3. Actividad 03 — Crear Consumidor de Eventos con .NET (Worker Service)

Objetivo: un microservicio independiente que escuche la cola pedidos_farmacia y procese cada pedido de forma asíncrona, sin bloquear al productor.

Parte A — Creación del proyecto

Se agregó a la misma solución un proyecto Worker Service llamado ServicioProcesadorPedidos, con el paquete NuGet RabbitMQ.Client instalado igual que en el productor.

Parte B — Modelo del evento

Se replicó el modelo PedidoCreadoEvent en Models/PedidoCreadoEvent.cs del consumidor. Debe coincidir exactamente en nombres y tipos con el del productor, porque ambos lados serializan/deserializan el mismo contrato JSON de forma independiente (no comparten código entre sí, como corresponde a microservicios reales).

Parte C — El consumidor (Worker.cs)

Worker.cs se conecta a RabbitMQ, declara la misma cola pedidos_farmacia y registra un consumidor asíncrono (AsyncEventingBasicConsumer) que se dispara cada vez que llega un mensaje:

```
var factory = new ConnectionFactory() { HostName = "localhost" };
var connection = await factory.CreateConnectionAsync();
var channel = await connection.CreateChannelAsync();

await channel.QueueDeclareAsync(
    queue: _queueName, durable: false, exclusive: false,
    autoDelete: false, arguments: null
);

var consumer = new AsyncEventingBasicConsumer(channel);
consumer.ReceivedAsync += async (sender, ea) =>
{
    var body = ea.Body.ToArray();
    var json = Encoding.UTF8.GetString(body);
    var evento = JsonSerializer.Deserialize<PedidoCreadoEvent>(json);
    ProcesarPedido(evento);
    await Task.Delay(500); // Simula carga de trabajo real
};

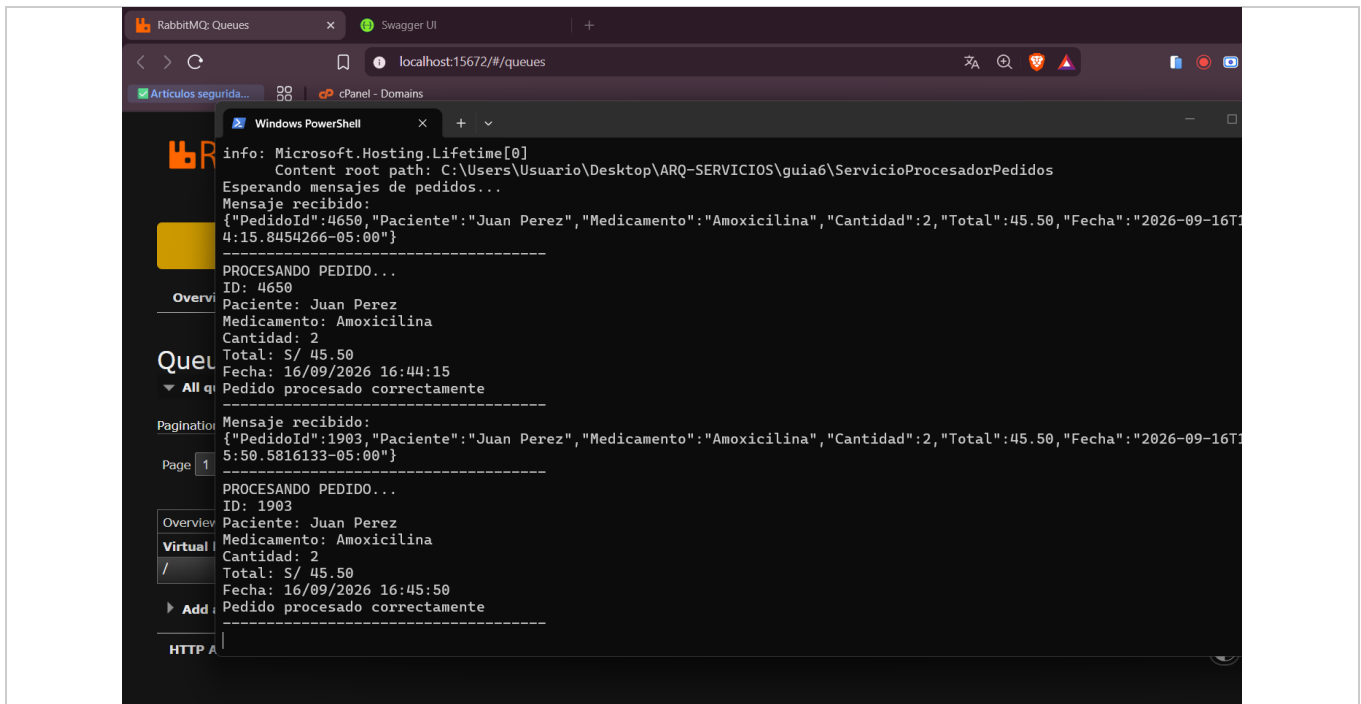
await channel.BasicConsumeAsync(
    queue: _queueName, autoAck: true, consumer: consumer
);
```

ProcesarPedido (la lógica de negocio del microservicio, aquí simulada con Console.WriteLine) imprime cada campo del pedido recibido y confirma su procesamiento. Todo el bloque de recepción está envuelto en try/catch para que un mensaje corrupto no tumbe al Worker completo.

Parte D — Prueba del sistema

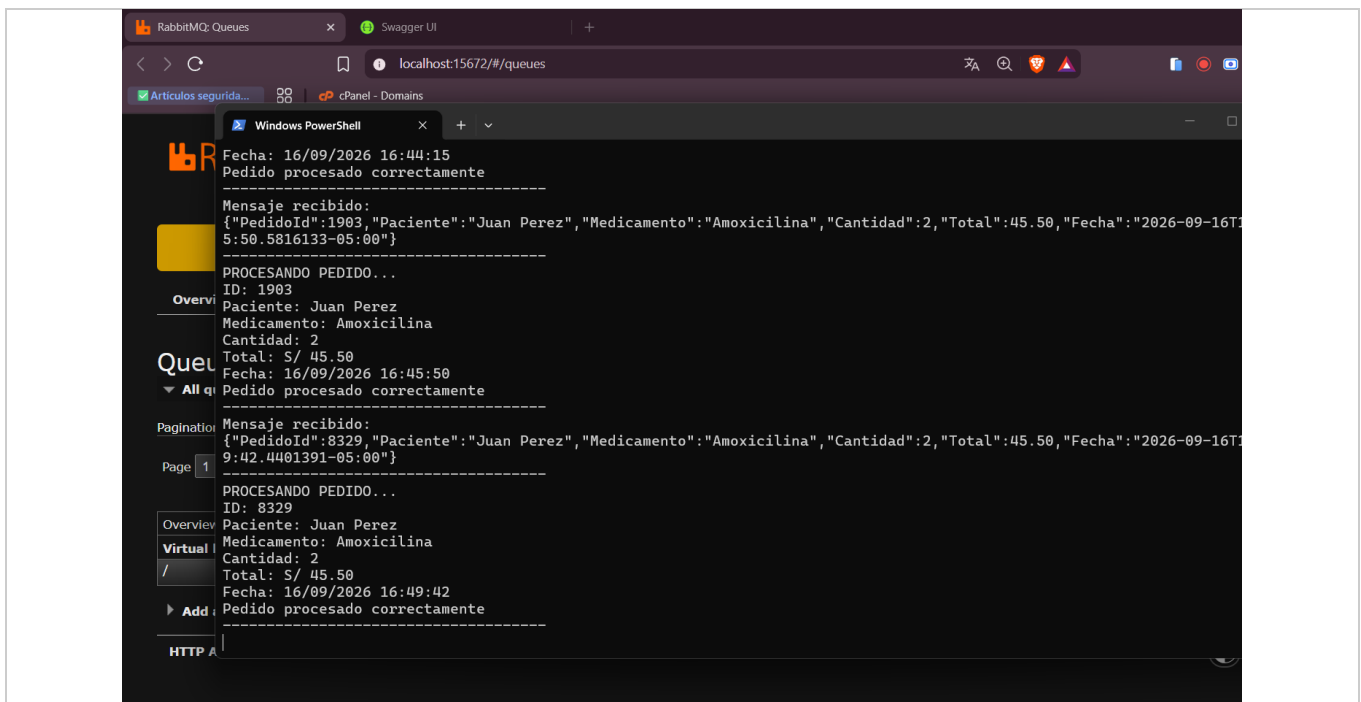
Se inició ServicioProcesadorPedidos con los 2 mensajes de la Actividad 02 todavía pendientes en la cola. El consumidor los recogió automáticamente en cuanto arrancó (sin necesidad de reenviar nada desde el productor), procesando primero el más antiguo (orden FIFO de la cola):

Evidencia — Consola de ServicioProcesadorPedidos — consumiendo los 2 pedidos que estaban pendientes en la cola desde la Actividad 02



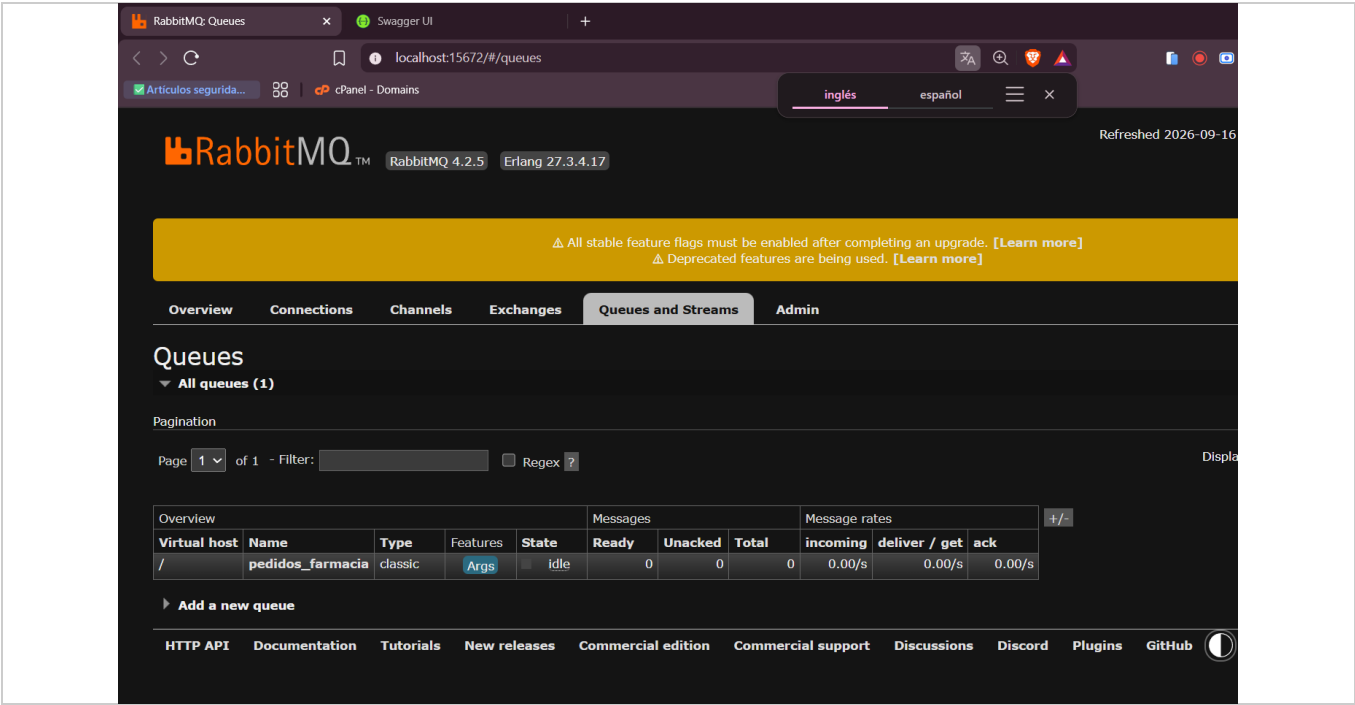
Después, con el consumidor ya corriendo, se generó un tercer pedido en caliente (POST /api/Pedidos/crear). El Worker lo recibió y procesó de inmediato, mostrando que el consumo ocurre en tiempo real y de forma asíncrona respecto al productor:

Evidencia — Consola de ServicioProcesadorPedidos — procesando en tiempo real un pedido nuevo (PedidoId 8329) apenas fue publicado



Verificación final en RabbitMQ: tras consumir los 3 mensajes, la cola pedidos_farmacia vuelve a 0 mensajes en Ready y 0 en total, confirmando que el consumidor los recibió y confirmó (autoAck) correctamente:

Evidencia — Panel de RabbitMQ — cola pedidos_farmacia en 0 mensajes tras ser consumidos por ServicioProcesadorPedidos



Producto esperado cumplido: microservicio consumidor independiente, que procesa los eventos de la cola de forma asíncrona (tanto los acumulados como los que llegan en tiempo real) sin bloquear al productor en ningún momento.

Cómo probar cada punto desde Visual Studio

Este proyecto se construyó con la CLI de .NET (`dotnet new / dotnet add package`), que es exactamente equivalente a los asistentes gráficos de Visual Studio. Para continuar trabajando o volver a probar cada actividad desde Visual Studio:

Preparar RabbitMQ antes de abrir Visual Studio

- Si instalaste RabbitMQ con el instalador de la guía: abre PowerShell como Administrador y ejecuta `net start RabbitMQ`.
- Si usas la build genérica (como en este informe): abre Git Bash, ve a la carpeta `rabbitmq_server-4.2.5/sbin` y ejecuta `./rabbitmq-server`, dejando esa ventana abierta mientras pruebas.
- Verifica en el navegador que `http://localhost:15672` carga (usuario/clave `guest/guest`) antes de tocar Visual Studio.

6.1 — Verificar la instalación

No requiere Visual Studio: basta con el navegador en `http://localhost:15672`. Si el panel carga y muestra la versión de RabbitMQ y Erlang, el punto está verificado.

6.2 — Probar el productor (ServicioPedidos)

- Abre `EcommerceMensajeria.slnx` en Visual Studio.
- En el Explorador de soluciones, clic derecho sobre `ServicioPedidos` → Establecer como proyecto de inicio (o usa "varios proyectos de inicio" si también quieres levantar el Worker al mismo tiempo).
- Presiona F5 (o `Ctrl+F5` para no depurar). Se abre Swagger en el navegador.
- En Swagger, expande `POST /api/Pedidos/crear` → Try it out → Execute. Debe responder 200 OK con un pedido generado.
- Repite el POST varias veces y confirma en `http://localhost:15672` → Queues and Streams que la cola `pedidos_farmacia` acumula un mensaje en Ready por cada ejecución.

6.3 — Probar el consumidor (ServicioProcesadorPedidos)

- Con `ServicioPedidos` ya corriendo (o con mensajes pendientes en la cola de una prueba anterior), clic derecho sobre `ServicioProcesadorPedidos` → Depurar → Iniciar nueva instancia (esto lo corre sin detener `ServicioPedidos`).
- Se abre una consola mostrando "Esperando mensajes de pedidos..." y de inmediato procesa cualquier mensaje pendiente en la cola.
- Con el Worker corriendo, ejecuta un nuevo `POST /api/Pedidos/crear` desde Swagger (o desde el cliente de consola si lo tienes de una guía anterior) y observa cómo aparece procesado en la consola del Worker en cuestión de milisegundos.
- Confirma en `http://localhost:15672` → Queues and Streams que Ready vuelve a 0 después de cada mensaje consumido.

Nota práctica: como en este informe, es más cómodo tener 3 ventanas visibles a la vez: RabbitMQ (navegador), `ServicioPedidos` (Swagger) y `ServicioProcesadorPedidos` (consola). Si Visual Studio solo permite depurar un proyecto de inicio a la vez, usa "Depurar → Iniciar nueva instancia" sobre el segundo proyecto en vez de cambiar el proyecto de inicio de la solución.